

MEMORIA ESCRITA DE SEQUENCE-FILTER

(Documentación técnica)

1. Resumen ejecutivo

Sequence-Filter es una aplicación web para búsqueda y filtrado de péptidos a partir de un dataset (FASTA o MySQL), combinando patrones de secuencia y propiedades fisicoquímicas (carga, punto isoeléctrico, % hidrofóbico, índice de Boman, momento hidrofóbico relativo). Posee una arquitectura con front-end estático (Nginx) + API Node/Express. Las secuencias se sirven preferentemente desde un FASTA cacheado en memoria; si no está disponible, se hace fallback a MySQL.

2. Objetivos

- Consultar y filtrar rápidamente una base de secuencias.
- Exportar resultados en CSV y FASTA.
- Mantener una UI responsiva y accesible con renderizado por lotes.
- Minimizar el “cold start”: precarga de FASTA en el servidor y cálculo on-demand de métricas en el cliente.

3. Alcance

- Búsqueda por patrón con comodín **X** → “cualquier AA [A–Z]”.
- Exclusiones múltiples (“Ignorar secuencia”) separadas por comas.
- Filtros numéricos por rango (mínimo/máximo).
- Estadísticos: conteo, % sobre total y AAs más frecuentes.
- Descargas y resaltado visual de coincidencias.

4. Arquitectura

- **Frontend (Nginx)**
 - HTML/CSS/JS estático ([frontend/*](#)).
 - Compresión gzip habilitada.
- **API (Node.js/Express)** ([api/server.js](#))
 - Endpoints REST para secuencias y health.
 - Carga de FASTA en arranque (warm-up) + cache en memoria.
 - Fallback a MySQL cuando FASTA no está disponible.
- **Base de datos**

- Preferencia: archivo FASTA ([api/sequences/BD_FINAL.fasta](#)).
- Alternativa: MySQL con autodetección de tabla/columnas.
- **Contenedores**
 - [docker-compose.yml](#) orquesta [api](#) y [frontend](#) en la red [inverpep_default](#).

5. Estructura del repositorio

Repositorio: <https://github.com/HinaraSM12/Sequence-filter>

```
Sequence-filter/
├── api/
│   ├── sequences/
│   │   └── BD_FINAL.fasta
│   ├── .dockerignore
│   ├── .env                # (opcional, en despliegue usar variables/secrets)
│   ├── db.js
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── assets/
│   │   └── img/
│   │       └── sequence-filter-logo.png
│   ├── app.js
│   ├── bioutils.js
│   ├── filter_seq.js
│   ├── index.html
│   ├── modal.js
│   └── styles.css
├── docker-compose.yml
├── nginx.conf
└── README.md
```

Descripción de cada archivo/carpeta

Raíz

- **docker-compose.yml**
Orquesta dos servicios: **api** (Node/Express) y **frontend** (Nginx) sobre la red [inverpep_default](#). Declara variables de entorno, healthchecks y volúmenes.
Importante: para pruebas sin proxy externo, publicar puertos ([8051:8051](#), [8080:80](#)). El healthcheck de la API usa [wget/curl](#); instálalo en la imagen o cambia el test.
- **nginx.conf**
Nginx sirve archivos estáticos y actúa como reverse proxy para [/api/](#) →

`http://api:8051` (sin / final para preservar el path). Activa gzip y políticas de caché (no-cache para HTML/JS/CSS; cache prolongada para assets).

Importante: puedes añadir cabeceras de seguridad (CSP, X-Content-Type-Options, etc.) si Nginx es el borde público.

- **README.md**

Guía de uso y despliegue, variables de entorno y flujo FASTA→MySQL. Conviene incluir comandos habituales y troubleshooting.

api/ (servicio Node/Express)

- **sequences/BD_FINAL.fasta**

Fuente de datos preferente. Se precarga en memoria al iniciar (warm-up) y se revalida por `mtime` para recarga automática si el archivo cambia.

- **.dockerignore**

Reduce el contexto de build. Excluir `.env`, `node_modules`, dumps, etc., acelera builds y evita filtrar secretos por error.

- **.env (local opcional)**

Configuración de API: `PORT`, `FASTA_PATH`, `FORCE_FASTA_ONLY`, credenciales y SSL de MySQL.

Importante: en producción usar variables/secrets del orquestador, no versionar `.env`.

- **db.js**

Pool `mysql2/promise` y helper `query`. Respeta el kill-switch `FORCE_FASTA_ONLY=true` (lanza `MYSQL_DISABLED`).

Importante: `DB_SSL=true` activa SSL laxo (`rejectUnauthorized:false`); para entornos regulados, cargar CA/certificados explícitos.

- **Dockerfile**

Construye la imagen de la API (Node 20 Alpine), instala dependencias y lanza `npm start`.

Importante: Alpine no trae `wget/curl`; ajusta el healthcheck o instala `curl`. Preferir `npm ci` + `package-lock.json` y `ENV NODE_ENV=production`.

- **package.json**

Metadatos y dependencias (`express`, `mysql2`, `cors`, `dotenv`). Script `start`.

Importante: mantener lockfile para builds reproducibles.

- **server.js**

Servidor Express con CORS y JSON, `trust proxy`.

- **FASTA cacheado** con selección inteligente de ruta y revalidación por `mtime`.
- **Fallback MySQL** si el FASTA no está disponible; autodetecta tabla y columnas (header/sequence) desde `information_schema`.
- **Normalización** de secuencias a `ACDEFGHIKLMNPQRSTVWY`.
- **Búsqueda por patrón** con `X → [A-Z]` y exclusiones múltiples (`ignore=...,...`); escapado de metacaracteres para evitar patrones peligrosos.
- Endpoints: `GET /api/health`, `GET /api/pingdb`, `GET /api/sequences` (+ alias `/sequences`).

Importante: límite `LIMIT 200000` en consultas MySQL; considerar paginación si el volumen crece.

frontend/ (SPA servida por Nginx)

- **index.html**
Estructura de la SPA: controles de filtro, estadísticas, tabla de resultados, descargas y modal de ayuda. Incluye meta no-cache para despliegues sin busting.
- **styles.css**
Estilos responsivos y tabla con contenedor desplazable. Define resaltado de coincidencias y estados del overlay.
- **app.js**
Lógica de interfaz: obtiene `/api/sequences`, aplica filtros y renderiza resultados.
 - **Renderizado por lotes** con `requestAnimationFrame` para evitar bloqueos.
 - **Métricas on-demand** con caché `Map` por secuencia normalizada.
 - Exporta resultados en CSV y FASTA desde el cliente.
 - RegExp segura: solo `X` se expande; resto de metacaracteres escapados.
Importante: el primer fetch transfiere la base completa; gzip ayuda pero la paginación del lado API sería recomendable en datasets muy grandes.
- **bioutils.js**
Utilidades bioquímicas: limpieza (`cleanSeq`, `cleanHeader`) y métricas (carga, pl por barrido 0–14 paso 0.01, % hidrofóbico, índice de Boman, momento hidrofóbico relativo).
Importante: todos los cálculos usan la secuencia limpia y se benefician del caché del `app.js`.
- **filter_seq.js**
Capa de compatibilidad que expone funciones en `window` para código legacy. Prescindible si todo consume `bioutils.js` directamente.
- **modal.js**
Control del modal (fade, cierre por Escape y clic en backdrop), bloqueo/desbloqueo de scroll. Mejorable con gestión de foco para accesibilidad.
- **assets/img/sequence-filter-logo.png**
Logo del encabezado. Nginx lo sirve con caché prolongada.

6. Flujo de datos

1. El usuario abre la SPA (Nginx sirve `index.html`, CSS y JS).
2. `app.js` llama a `GET /api/sequences` para cargar la base (FASTA o MySQL).
3. El usuario ingresa patrón/filtros → `app.js` filtra en memoria:
 - Patrones/ignorar vía RegExp (`X` → `[A-Z]`).
 - Métricas se calculan on-demand y se cachean en `Map`.
 - Renderizado por lotes para evitar bloquear la UI.
4. Exportación CSV/FASTA se arma en el cliente.

7. API

GET /api/health

- Respuesta: `{ ok: true }`

GET /api/pingdb

- Prueba de conectividad MySQL (si habilitado).
- Respuesta: { ok: true, db: { ok: 1 } } o 500.

GET /api/sequences?pattern=KAXKA&ignore=GG,PPP

- **Preferencia:** leer FASTA cacheado.
- **Fallback:** MySQL con autodetección de tabla/columnas si no está configurado via `TABLE_NAME`, `HEADER_COLUMN`, `SEQUENCE_COLUMN`.
- Respuesta: [{ header, sequence }] normalizados (AAs válidos ACDEFGHIKLMNPQRSTVWY).

Normalización de patrones

- Se escapan metacaracteres y **X** se expande a `[A-Z]`.
- `ignore` acepta múltiples patrones separados por coma.

8. Configuración (variables de entorno)

API (`api/.env` o `docker-compose.yml`):

- `PORT=8051`
- `FASTA_PATH=/app/sequences/BD_FINAL.fasta`
- `FORCE_FASTA_ONLY=false` (si `true`, deshabilita MySQL)
- `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, `DB_SSL`
- (Opcional) `TABLE_NAME`, `HEADER_COLUMN`, `SEQUENCE_COLUMN`

Ejemplo:

```
# API
PORT=8051

# DB remota

DB_HOST=sisifo.medellin.unal.edu.co

DB_PORT=3306

DB_DATABASE=prospeccionydis

DB_USERNAME=prospeccionydis

DB_PASSWORD=biomole1520

DB_SSL=false
```

```
# autodetección (deja vacíos si quieres autodetectar)

TABLE_NAME=

HEADER_COLUMN=

SEQUENCE_COLUMN=

# Rutas FASTA

FASTA_PATH=/app/sequences/BD_FINAL.fasta

# Permitir fallback a MySQL si el FASTA falla

FORCE_FASTA_ONLY=false
```

Nginx:

- VIRTUAL_HOST, LETSENCRYPT_HOST, LETSENCRYPT_EMAIL, VIRTUAL_PORT

9. Despliegue

```
# En la raíz del proyecto (nivel de docker-compose.yml)
docker compose build --no-cache api
docker compose up -d
```

```
# Ver logs
docker compose logs -f api
docker compose logs -f frontend
```

10. Rendimiento y decisiones

- **Warm-up FASTA** al iniciar el servidor → elimina latencia del primer hit a `/api/sequences`.
- **Cálculo on-demand** de métricas en el cliente → UI responsiva al cargar.
- **Renderizado por lotes** y `requestAnimationFrame` para no bloquear el hilo.
- **gzip** en Nginx para JSON/HTML/CSS/JS.
- **Cache en Map** para métricas por secuencia (clave: secuencia limpia).

11. Seguridad

- `trust proxy` habilitado para Nginx.
- No se persisten datos del usuario.
- Sanitización fuerte de secuencias y cabeceras.

- En producción, servir por HTTPS (Let's Encrypt con tu reverse proxy/stack).

12. Respaldo y restauración

- **FASTA**: versionar `api/sequences/BD_FINAL.fasta`.
- **MySQL** (si se usa): dumps regulares (`mysqldump`).
- Nginx y API son **stateless** → se reconstruyen con `docker compose build`.

13. Mantenimiento

- Reemplazo de dataset: actualizar `BD_FINAL.fasta` y redeploy.
- Actualizaciones: bump de Node/npm y rebuild de imagen API.
- Monitoreo: `GET /health` y `GET /api/health` (healthchecks de Docker).

14. Limitaciones conocidas

- La consulta inicial transfiere toda la base en JSON (optimizado con gzip).
- Los cálculos de métricas por secuencia son $O(n)$ (cacheados).

15. Licencia y autores

Licencia

Este proyecto se distribuye bajo la **Licencia MIT** (permisiva). Puedes usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del software, siempre que mantengas el aviso de copyright y esta nota de licencia en las copias o partes sustanciales del software. No ofrece garantías.

Autores y créditos

- **Autora principal:** *Hinara Pastora Sánchez Mata*. Información de contacto: 3044822338, hisanchezm@unal.edu.co
- **Crédito institucional:** “Desarrollado por la línea de interacciones moleculares — **Biología Funcional**”.